

Machine Learning

5. *Supervised Learning 2*

Sunwoo Lee
sunwool@inha.ac.kr



INHA UNIVERSITY

Today, we will learn...

- Supervised Learning 2
 - What is decision tree?
 - How is it built?
 - Overfitting Issue



Do not share or distribute these slides!

저작권 안내 / Copyright Notice

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. **약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다. (일부 공개로 제공되는 유튜브 강의를 불법으로 공유하거나 배포하는 행위를 금함)**

The teaching materials provided on this site are copyrighted and signed legally by the Korea Reproduction and Transmission Rights Association. Using these materials beyond the scope of the agreement is a violation against copyright laws. **Hence, one is not allowed to open or share the materials in public. Using them outside of the class is strictly prohibited.**

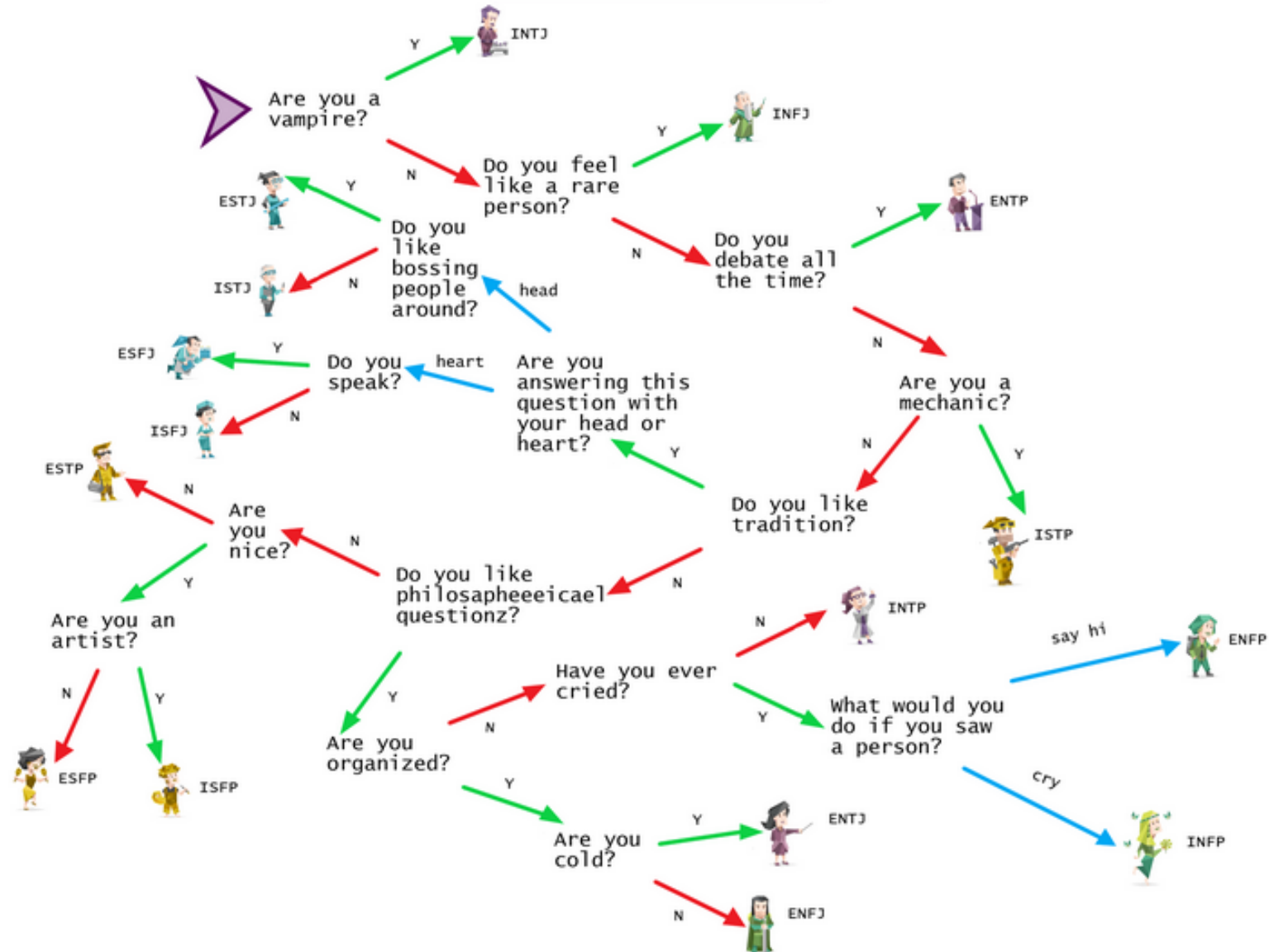
2020. 3. 11.

인하대학교·한국복제전송저작권협회

Inha University · Korea Reproduction and Transmission Rights Association

- ***Decision Tree***
- Building a Decision Tree
- Overfitting Issue

16 personalities test be like



Function Approximation

- **Problem Setting**

- A set of independent variables (observations) x
- A set of dependent variables (labels) y
- Unknown target function $f: x \rightarrow y$
- A set of function hypothesis $H = \{h|h: x \rightarrow y\}$

종속변수 y 에 대해서 x 가 들어왔을 때 어떤 y 를 매칭시킬 것인가?

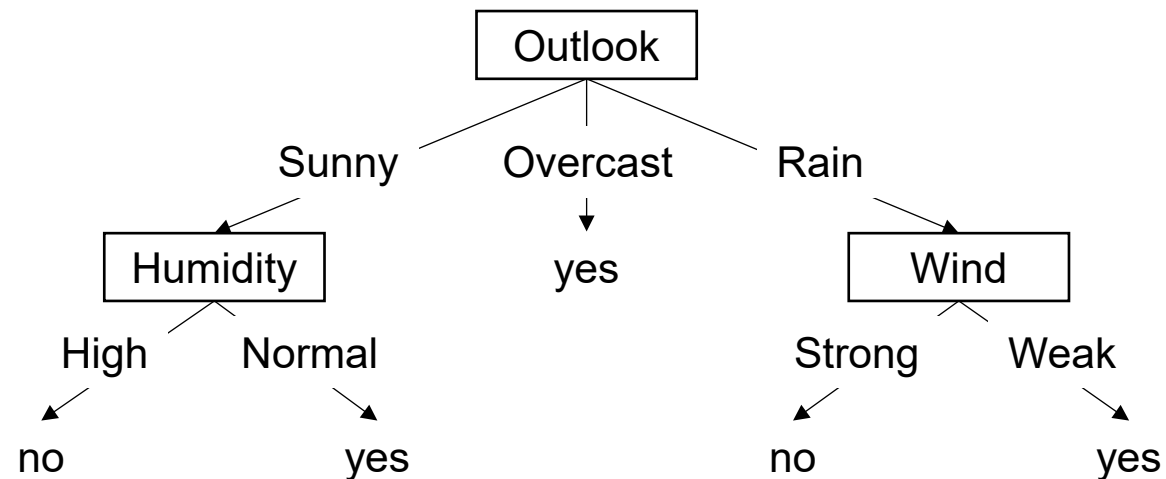
- **Input:** $\{\langle x_i, y_i \rangle\}_{i=1}^n = \{\langle x_1, y_1 \rangle, \langle x_2, y_2 \rangle, \dots, \langle x_n, y_n \rangle\}$
- **Output:** Hypothesis $h \in H$ that best approximates f .

Our goal is to find a hypothesis h that is **very similar** to the target function f .

가능한 해답 중에서 함수 $f(x \rightarrow y)$ 에 가장 가까운 가설 h 를 선택한다

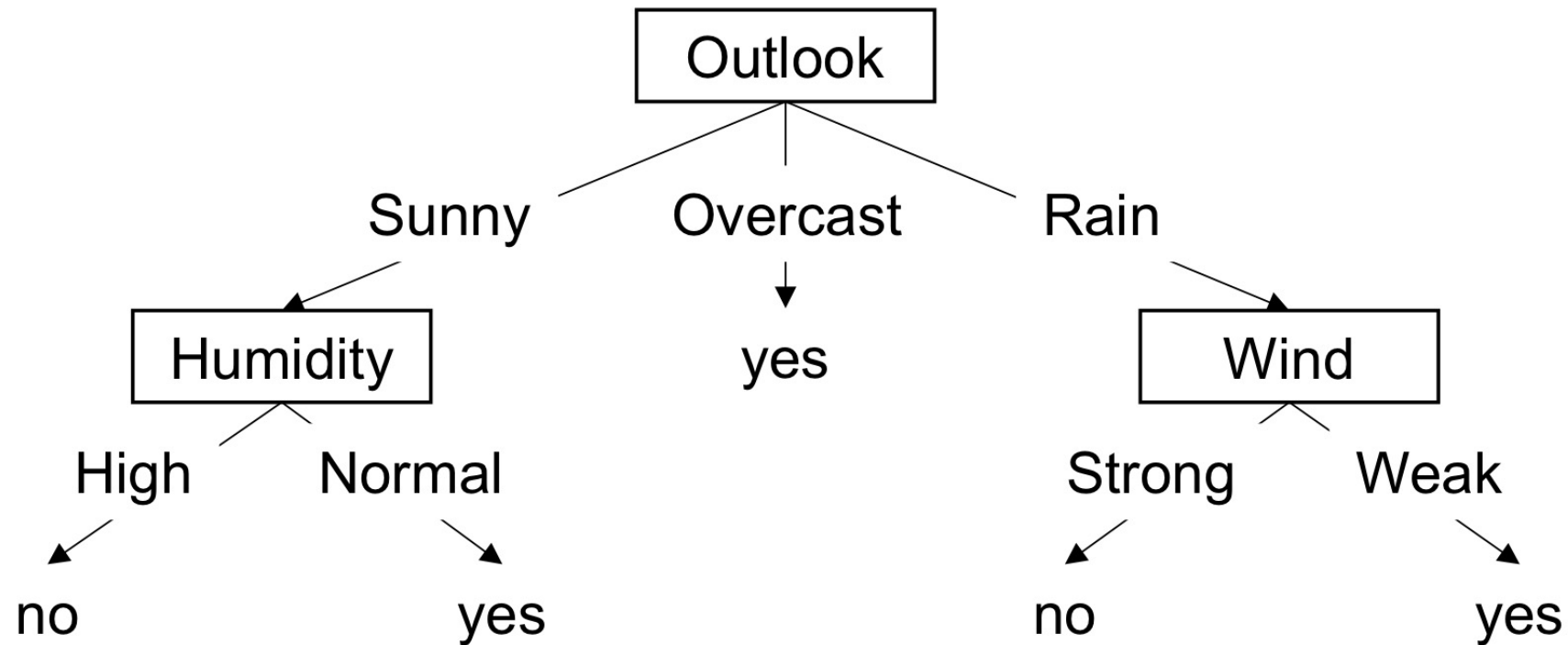
Decision Tree (1/4)

- Decision trees are a non-parametric supervised learning method used for classification / regression.
- The goal is to create a model that predicts the value of a target variable ***by learning decision rules inferred from the data*** features.



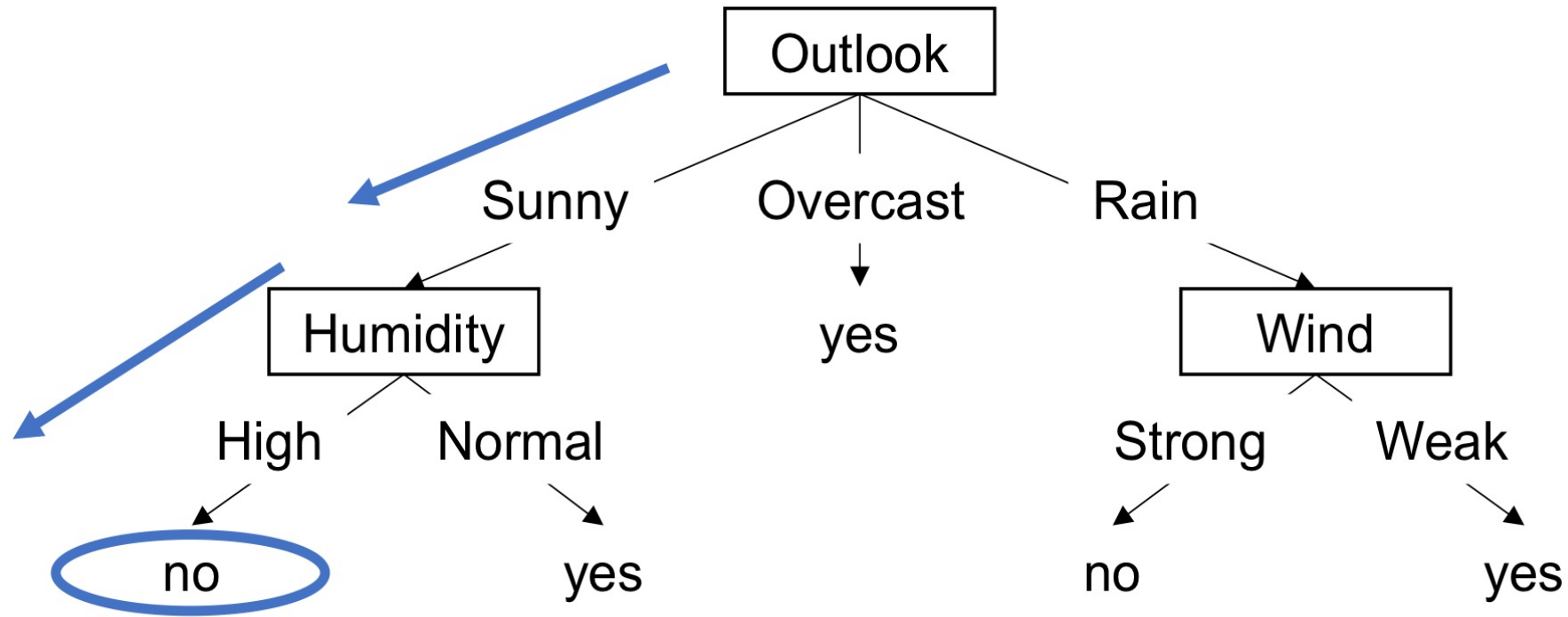
Decision Tree (2/4)

- Each internal node: test one attribute x_i .
- Each branch from a node: selects one value for x_i .
- Each leaf node: predict y (or $p(y|x \in \text{leaves})$).



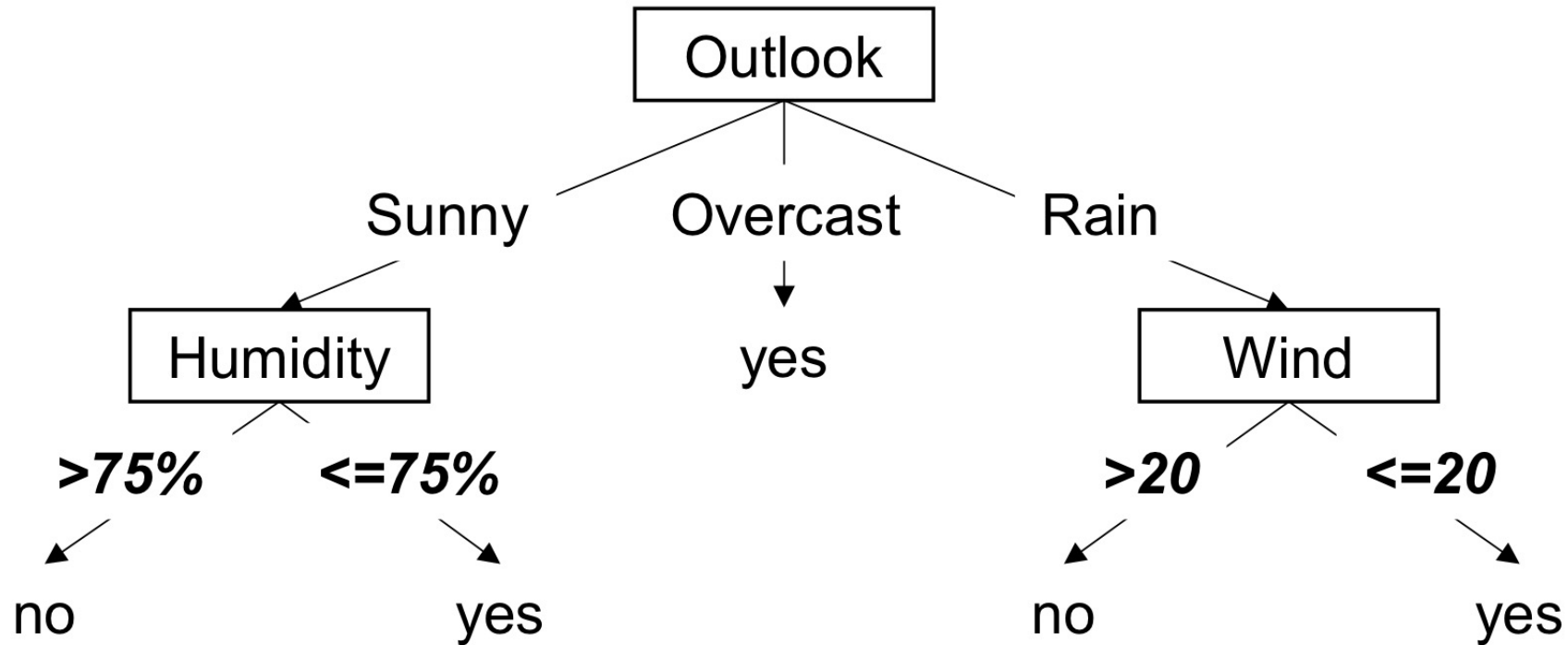
Decision Tree (3/4)

- Example: What prediction would our model make for:
 - <outlook=sunny, temperature=hot, humidity=high, wind=weak>?



Decision Tree (4/4)

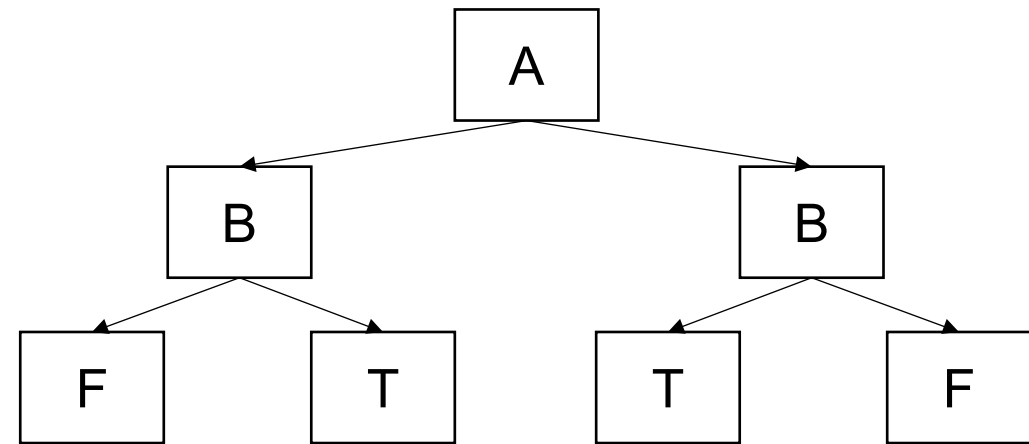
- If features are continuous, internal nodes can test the value of the feature against a threshold.



Expressiveness

- Decision trees can represent any Boolean function of the input attributes.
- However, in the worst case, the tree will require exponentially many nodes.

A	B	A xor B
T	T	F
T	F	T
F	T	T
F	F	F



- Decision Tree
- ***Building a Decision Tree***
- Overfitting Issue

Attribute Selection at Each Node

- How should we choose which attribute to split a given data set?
 - To make the tree compact.
 - To make the tree accurate.
- Some possibilities are:
 - Random: random attribute...
 - Least-Values: attribute with the smallest # of possible values
 - Most-Values: attribute with the largest # of possible values
 - ***Customized criterion: attribute with a certain good criterion***

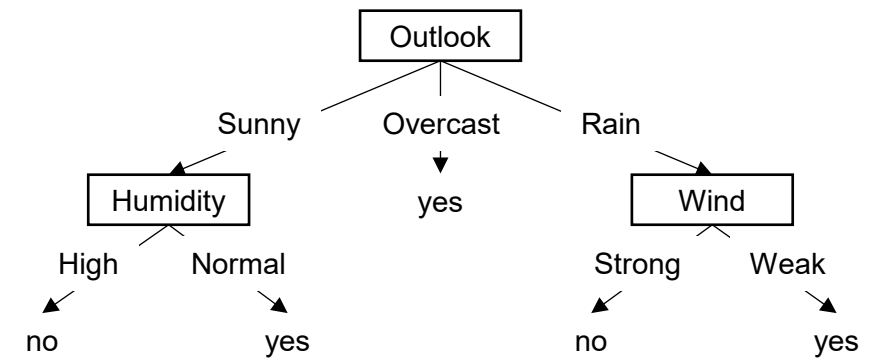


Algorithms

- Many algorithms have been proposed so far:
 - CART (Classification and Regression Trees)
 - **ID3** (Iterative Dichotomiser 3)
 - C4.5
 - CHAID (Chi-squared Automatic Interaction Detection)
- We will cover ID3 in this class.

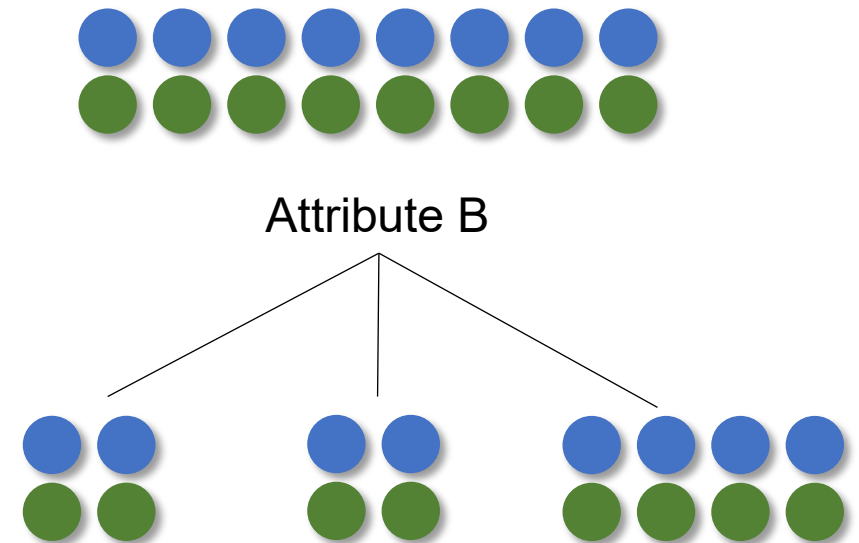
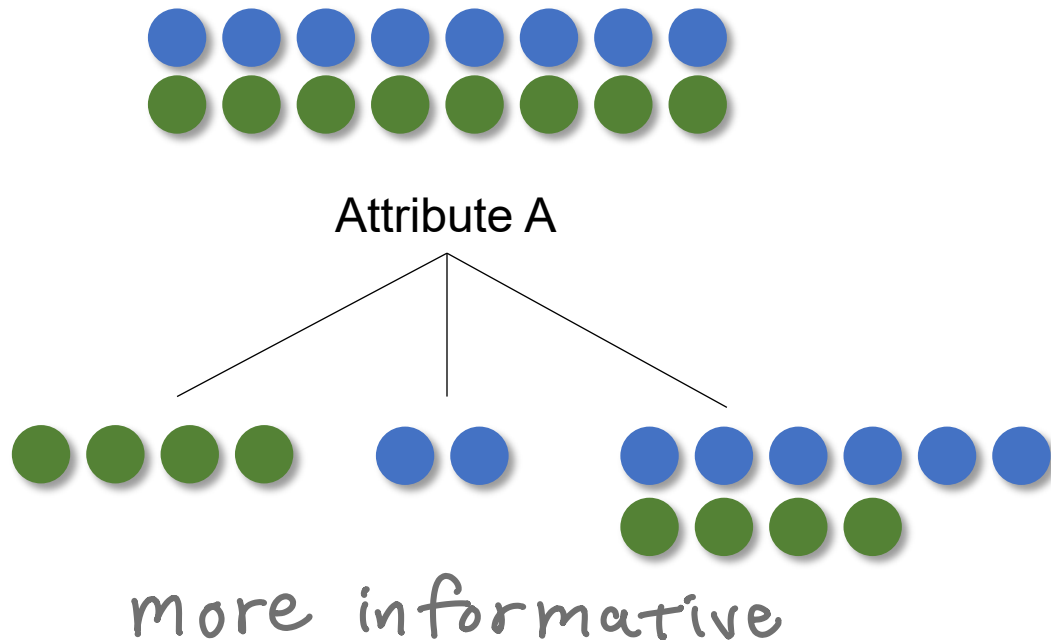
ID3 (Iterative Dichotomiser 3)

- ID3 is a top-down greedy approach to build a tree.
- Problem Settings
 - Set of possible instances (observations) X .
 - Each instance x in X is a feature vector.
 - E.g., $\langle \text{Humidity}=\text{low}, \text{Wind}=\text{weak}, \text{Outlook}=\text{rain}, \text{Temperature}=\text{hot} \rangle$
 - Unknown target function $f: X \rightarrow y$.
 - Set of function hypotheses $H = \{h | h: X \rightarrow y\}$.
 - Each hypothesis h is a decision tree.
 - The trees sort X to a leaf, which assigns y .



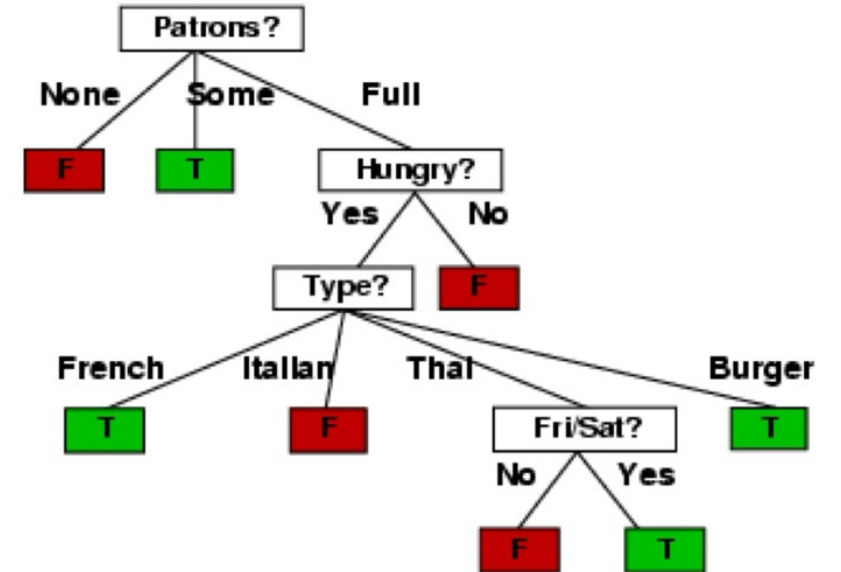
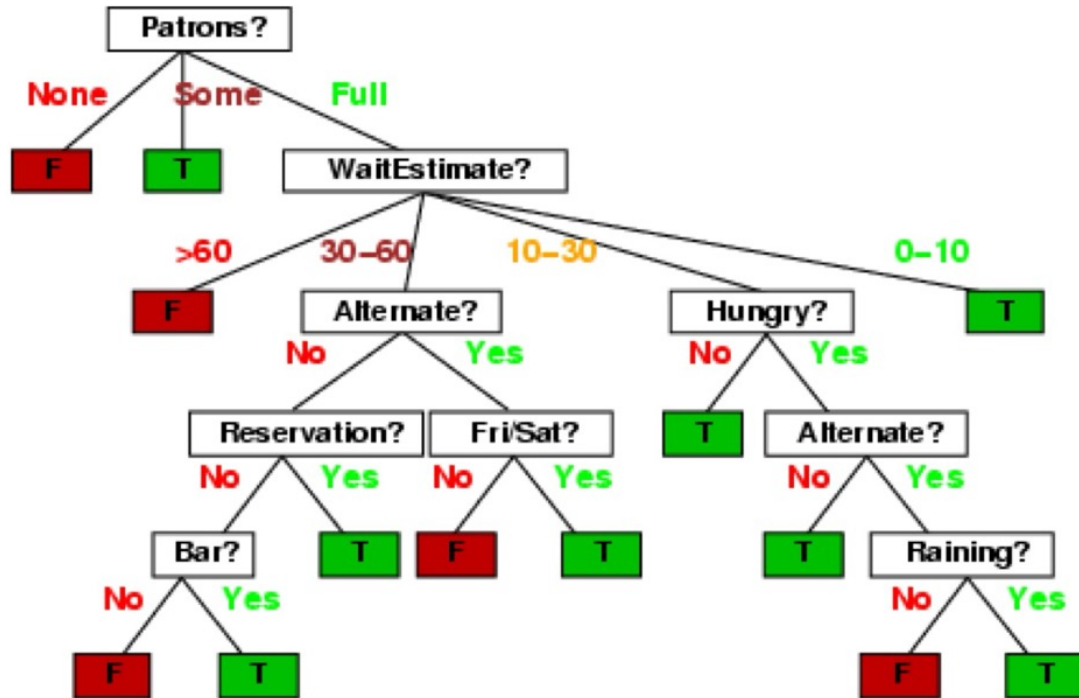
Choosing an Attribute at Each Node

- **Idea:** a good attribute splits the examples into subsets that are (ideally) 'all positive' or 'all negative'.



Which attribute is more informative?

Which tree is the better one?



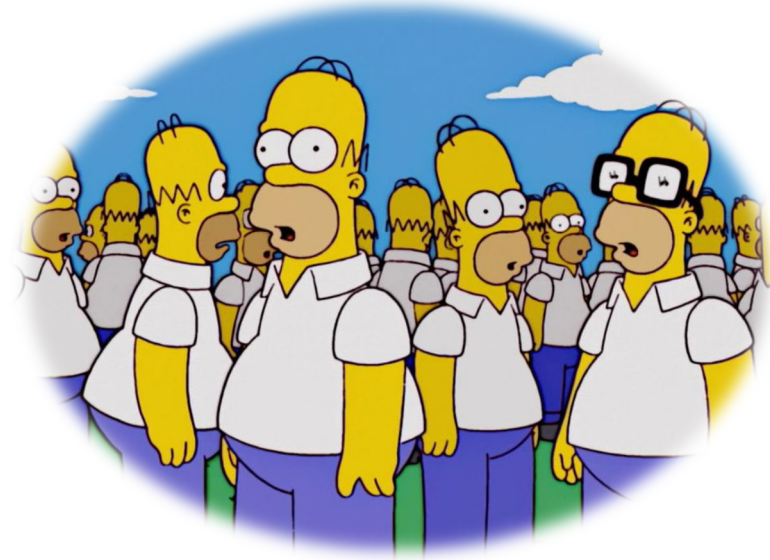
more compact

Impurity

- Impurity / Entropy (informal) measures the level of impurity in a group of examples.



Impure group



Pure group

Entropy

- Entropy of a random variable x : the average level of information (uncertainty).

$$H(x) = - \sum_{i=1}^n p(x = i) \log_2 p(x = i)$$

n is the number of training data (possible values of x).

X가 특정 값일 확률의 기댓값

=> 높으면, 각 확률의 값이 서로 비슷하다 = 해당 노드의 정보가 적다

=> 낮으면, 특정 확률의 값이 0에 가깝다 = 해당 노드의 정보가 많다

Information Gain (1/2)

- We want to **determine which attribute** in a given set of training features is **most useful** for discriminating between classes.
- Information Gain (IG) is a metric that quantifies such 'usefulness'.
- IG is the entropy difference between before and after splitting the node.
- **By choosing an attribute with the highest IG, the entropy of the whole tree can be minimized.**

Information Gain (2/2)

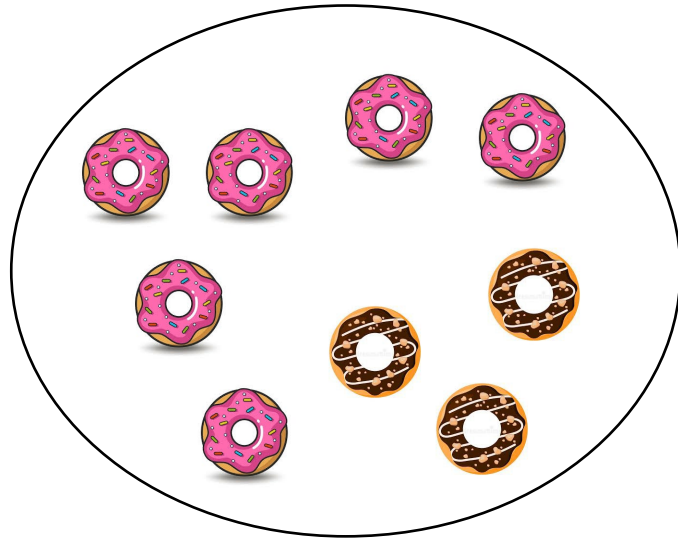
- Let T denotes a set of training data, each of the form $\langle x, y \rangle = \langle x_1, x_2, \dots, x_k, y \rangle$ where $x_a \in \text{values}(a)$ is the value of a^{th} attribute.
- For a value v taken by attribute a , let $S_a(v) = \{x \in T | x_a = v\}$ be defined as the set of training data that has $a = v$.
- Then, the conditional entropy is:

$$H(T|a) = \sum_{v \in \text{values}(a)} p(a = v) \cdot H(S_a(v)).$$

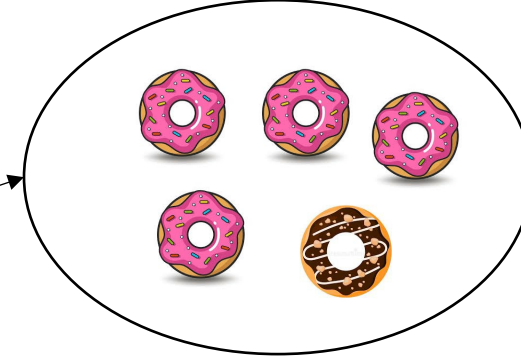
- Information Gain (IG) is defined as:

$$IG(T, a) = H(T) - H(T|a)$$

Entire population: T (9 donuts)

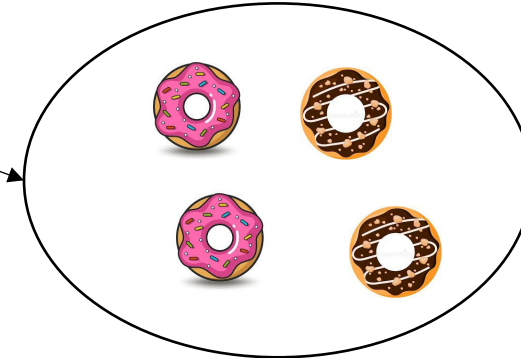


$a = 0$



$$\begin{aligned} \text{Child entropy } H(T|a = 0) &= -\left(\frac{4}{5} \log_2 \frac{4}{5}\right) - \left(\frac{1}{5} \log_2 \frac{1}{5}\right) \\ &= 0.25754 + 0.46439 = \mathbf{0.72193} \end{aligned}$$

$a = 1$



$$\begin{aligned} \text{Parent entropy } H(T) &= -\left(\frac{6}{9} \log_2 \frac{6}{9}\right) - \left(\frac{3}{9} \log_2 \frac{3}{9}\right) \\ &= 0.38998 + 0.52832 = \mathbf{0.9183} \end{aligned}$$

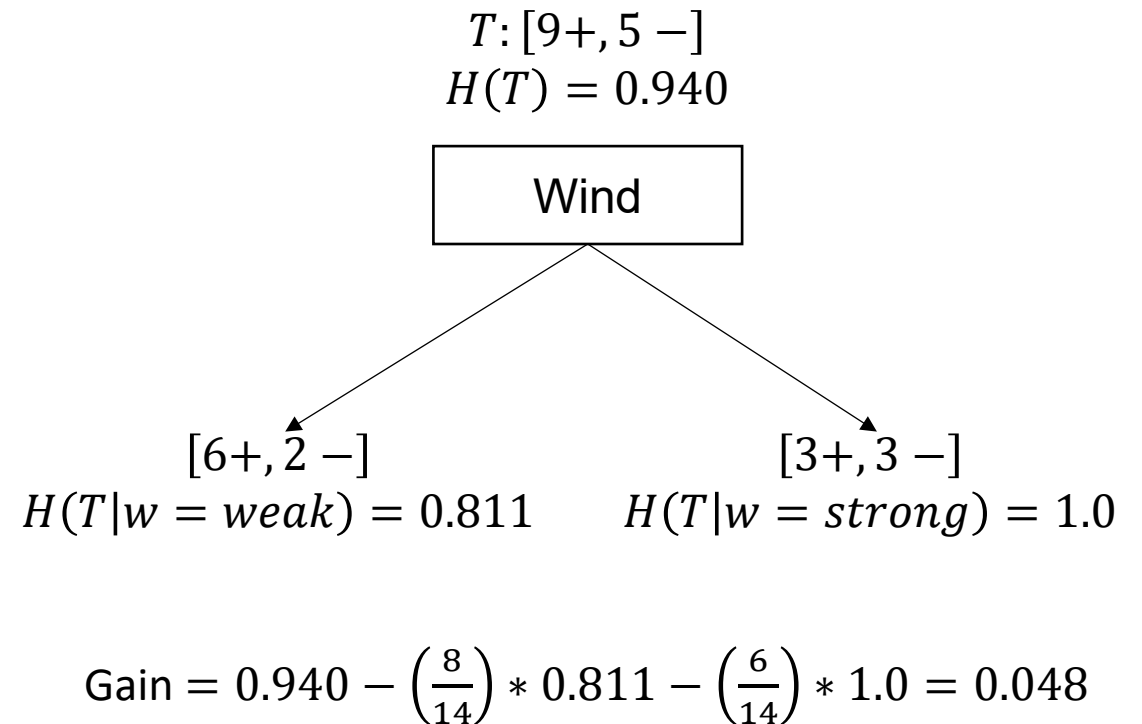
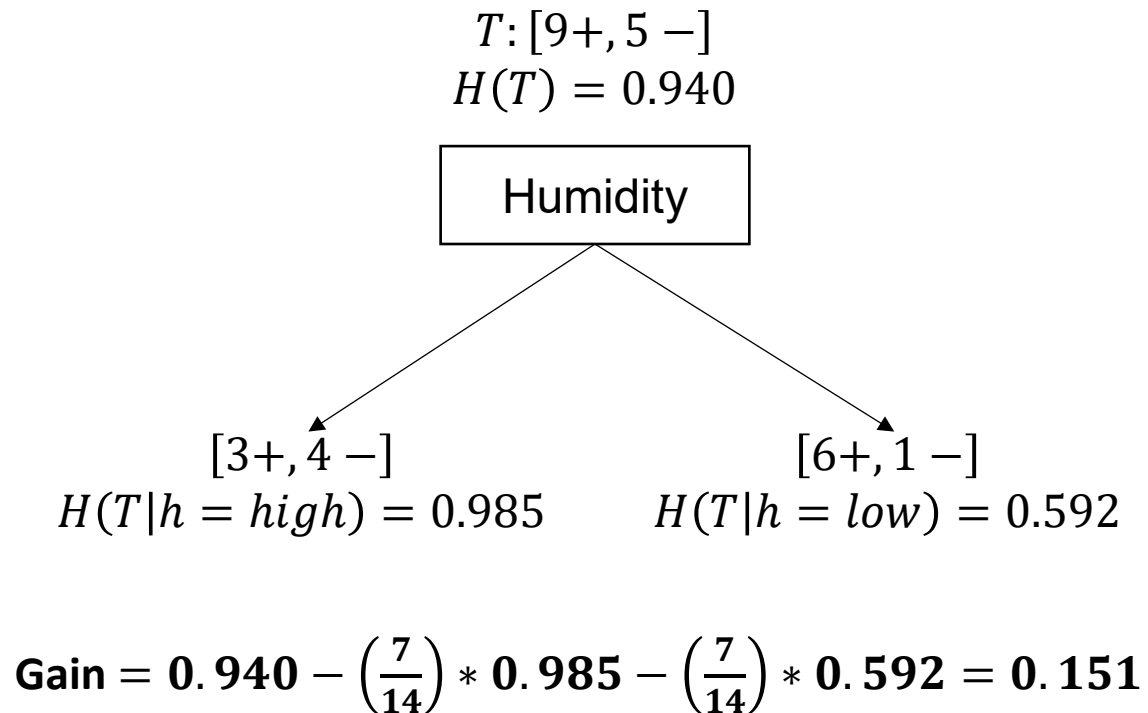
$$\begin{aligned} \text{Child entropy } H(T|a = 1) &= -\left(\frac{2}{4} \log_2 \frac{2}{4}\right) - \left(\frac{2}{4} \log_2 \frac{2}{4}\right) \\ &= 0.5 + 0.5 = \mathbf{1} \end{aligned}$$

$$\begin{aligned} H(T|a) &= p(a = 0)H(T, a = 0) + p(a = 1)H(T, a = 1) \\ &= \frac{5}{9} \cdot 0.72193 + \frac{4}{9} \cdot 1 = 0.401072 + 0.444444 = 0.845517 \end{aligned}$$

$$\text{Information gain } IG(T, a) = H(T) - H(T|a) = 0.9183 - 0.845517 = \mathbf{0.072783}$$

Selecting the Next Attribute Based on IG

- Which attribute is a better one?



Decision Tree Building (Learning)

Algorithm: ID3

1. Assign all training instances to the root of the tree and set current node to root node.
2. Choose one non-leaf node and find an attribute that gives the best Information Gain for the chosen node.
 - If the best Information Gain is 0, set the current node to be a leaf and repeat step 2.
3. Return the tree.

Sample Dataset: Tennis Game Records

- Columns denote features X .
- Rows denote labeled instances $\langle x_i, y_i \rangle$.
- Class label denotes whether a tennis game was played or not.

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Choosing The First Attribute

- First, put all the data samples in the root node.
- Then, first calculate the entropy before the first split as follows.

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

$$\begin{aligned}
 H(Play) &= - \sum_{i=1}^c p_i \log_2(p_i) \\
 &= - \left(\frac{5}{14} \log_2 \left(\frac{5}{14} \right) + \frac{9}{14} \log_2 \left(\frac{9}{14} \right) \right) \\
 &= 0.94
 \end{aligned}$$

Check Outlook Attribute

- Now, let us find the best attribute among 'outlook', 'temp', 'humidity', and 'wind'.

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

$$\begin{aligned}
 H(\text{Play}, \text{Outlook}) &= \sum_{i=1}^c p_i H(\text{Play} | \text{Outlook} = i) \\
 &= p(\text{sunny}) \times H(\text{Play} | \text{Outlook} = \text{sunny}) \\
 &\quad + p(\text{overcast}) \times H(\text{Play} | \text{Outlook} = \text{overcast}) \\
 &\quad + p(\text{rain}) \times H(\text{Play} | \text{Outlook} = \text{rain}) \\
 &= \frac{5}{14} \left(-\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) \right) \\
 &\quad + \frac{4}{14} \left(-\frac{4}{4} \log_2 \left(\frac{4}{4} \right) - \frac{0}{4} \log_2 \left(\frac{0}{4} \right) \right) \\
 &\quad + \frac{5}{14} \left(-\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) \right) = 0.6935
 \end{aligned}$$

Check Temperature Attribute

$$H(Play, Temp) = \sum_{i=1}^c p_i H(Play|Temp = i)$$

$$= p(hot) \times H(Play|Temp = hot) \\ + p(mild) \times H(Play|Temp = mild) \\ + p(cool) \times H(Play|Temp = cool)$$

$$= \frac{4}{14} \left(-\frac{2}{4} \log_2 \left(\frac{2}{4} \right) - \frac{2}{4} \log_2 \left(\frac{2}{4} \right) \right) \\ + \frac{6}{14} \left(-\frac{4}{6} \log_2 \left(\frac{4}{6} \right) - \frac{2}{6} \log_2 \left(\frac{2}{6} \right) \right) \\ + \frac{4}{14} \left(-\frac{3}{4} \log_2 \left(\frac{3}{4} \right) - \frac{1}{4} \log_2 \left(\frac{1}{4} \right) \right) = 0.911$$

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Check Humidity Attribute

$$H(Play, Humidity) = \sum_{i=1}^c p_i H(Play|Humidity = i)$$

$$= p(high) \times H(Play|Humidity = high) + p(normal) \times H(Play|Humidity = normal)$$

$$= \frac{7}{14} \left(-\frac{3}{7} \log_2 \left(\frac{3}{7} \right) - \frac{4}{7} \log_2 \left(\frac{4}{7} \right) \right) + \frac{7}{14} \left(-\frac{6}{7} \log_2 \left(\frac{6}{7} \right) - \frac{1}{7} \log_2 \left(\frac{1}{7} \right) \right) = 0.7884$$

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Check Windy Attribute

$$H(Play, Windy) = \sum_{i=1}^c p_i H(Play|Windy = i)$$

$$= p(weak) \times H(Play|Windy = weak) + p(strong) \times H(Play|windy = strong)$$

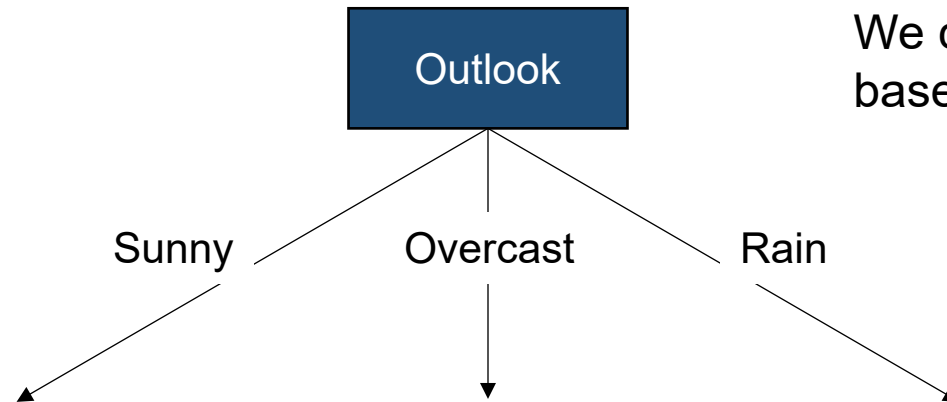
$$= \frac{8}{14} \left(-\frac{6}{8} \log_2 \left(\frac{6}{8} \right) - \frac{2}{8} \log_2 \left(\frac{2}{8} \right) \right) + \frac{6}{14} \left(-\frac{3}{6} \log_2 \left(\frac{3}{6} \right) - \frac{3}{6} \log_2 \left(\frac{3}{6} \right) \right) = 0.8921$$

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Choose The Attribute with The Greatest IG

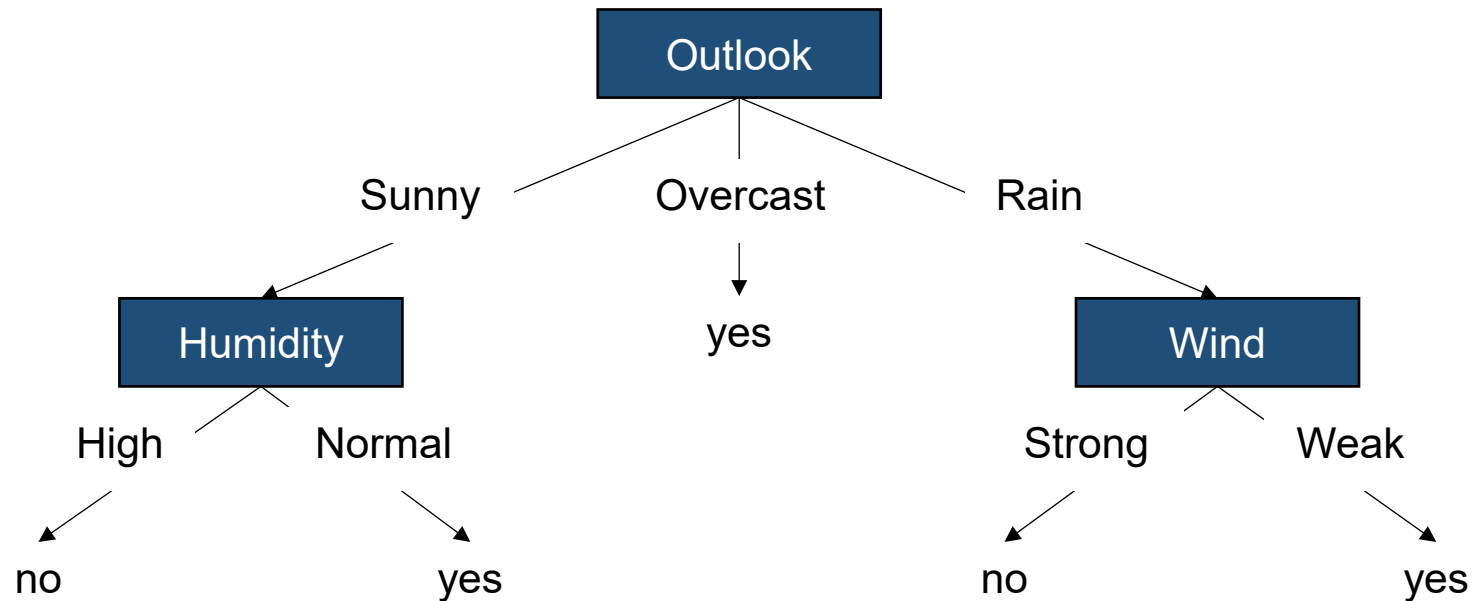
- $H(Play) - H(Play, Outlook) = 0.94 - 0.6935 = 0.25$
- $H(Play) - H(Play, Temp) = 0.94 - 0.911 = 0.03$
- $H(Play) - H(Play, Humidity) = 0.94 - 0.7884 = 0.1514$
- $H(Play) - H(Play, Windy) = 0.94 - 0.8921 = 0.047$



We choose 'outlook' and split the data based on its attribute values.

Repeat The Previous Steps...

- Until all the nodes are either a leaf node or split.
 - Note that all the attributes are not necessarily used to build the tree.
 - The ID3 algorithm returns quite a concise tree (only depth of 2).



Pros & Cons

- **Pros:** Information Gain-based tree building approach is automatic and guarantees to minimize the entropy at all the nodes.
↳ 트리의 Depth가 낮아짐
- **Cons:** The algorithm prefers attributes with a large number of values that split the data into small and pure subsets (fat trees).
 - E.g., when you build a tree for categorizing the customer patterns, you may not want to rely on the customer ID that most likely has the largest number of possible values... 엔트로피 측면에서는 최적이거나 분류의 의미가 탐색됨

Information Gain Ratio (1/2)

- Information Gain Ratio is later proposed to address the issue, which normalizes the information gain based on the group size of each child. 선택지가 많아지는 branching이 패널티 부여

- Split Information

$$SplitInfo(T, a) = - \sum_{v \in values(a)} \frac{|T_v|}{|T|} \log_2 \frac{|T_v|}{|T|},$$

where T_v is a subset of T for which a has value of v .

- Information Gain Ratio (IGR)

$$IGR(T, a) = \frac{IG(T, a)}{SplitInfo(T, a)}$$

← The attribute with a large number of possible values has a large value of it.

Split Information (SI)

이 편향을 보정하기 위한 “패널티 항”이 바로 **Split Information**입니다. [youtube](#)

$$SplitInfo(T, a) = - \sum_{v \in values(a)} \frac{|T_v|}{|T|} \log_2 \frac{|T_v|}{|T|}$$

여기서 T_v 는 속성 a 의 값이 v 인 샘플들의 집합이고, $|T|$ 는 전체 샘플 수입니다 [첨부 파일](#) . 이 공식의 구조는 엔트로피와 동일한 형태입니다. 핵심은, 분기가 얼마나 균등하게 또는 잘게 쪼개지는지를 측정한다는 것입니다. 속성의 값의 종류가 많을수록, 혹은 분기 후 각 집합의 크기가 균등할수록 SI 값이 커집니다 [arxiv](#) .

예시로 직관을 얻어 봅시다:

- 속성 **A**가 2가지 값으로 데이터를 **50:50**으로 나눈다면, $SI = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1.0$
- 속성 **B**가 4가지 값으로 데이터를 **25:25:25:25**로 나눈다면, $SI = -4 \times 0.25 \log_2 0.25 = 2.0$

속성 B처럼 가지가 많이 생길수록 SI가 커지고, 결국 IGR이 낮아지는 패널티를 받게 됩니다. [tistory](#)

Information Gain Ratio (IGR)

최종적으로 IGR은 다음과 같이 정의됩니다: [첨부 파일](#)

$$IGR(T, a) = \frac{IG(T, a)}{SplitInfo(T, a)}$$

분자의 IG는 해당 속성으로 분기했을 때 불순도가 얼마나 줄었는지를 나타내고, 분모의 SI는 그 분기가 얼마나 “잘게 쪼개지는” 분기였는지를 나타냅니다. 즉, 같은 **IG**를 얻더라도 가지를 적게 만드는 속성이 더 높은 **IGR**을 받게 됩니다. 슬라이드의 오른쪽 주석처럼, 가능한 값의 수가 많은 속성일수록 SI가 커져 IGR이 낮아집니다.

Information Gain Ratio (2/3)

- **Pros:**

- Penalize the attributes that excessively creates branches.
- Consequently, trees tend to achieve better generalization.
- It also likely reduces the model complexity.

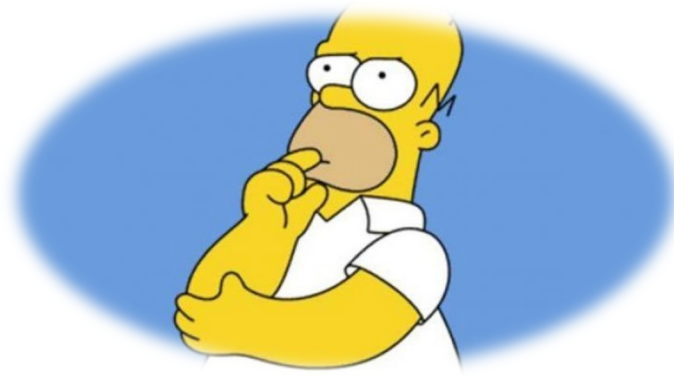
- **Cons:**

- Additional computations.
- Sometimes, it over-penalizes the critical attributes, degrading the accuracy.
- ***Biased decisions*** (it prefers less branches).

└─> 최적 선택지 개수를 결정하기 어려움, 따라서 경우에 따라 다른 방법을 택해야 함

Information Gain Ratio (2/2)

- *Information Gain Ratio* makes the tree biased towards the attributes with a small number of possible values.
- That is, if a problem consists of many attributes with a large number of possible values, they will mostly appear in the bottom of the tree (biased solution sometimes hurts the performance).
- C4.5 algorithm tackles this issue by having two step attribute selection schemes.



Which Tree Should We Build?

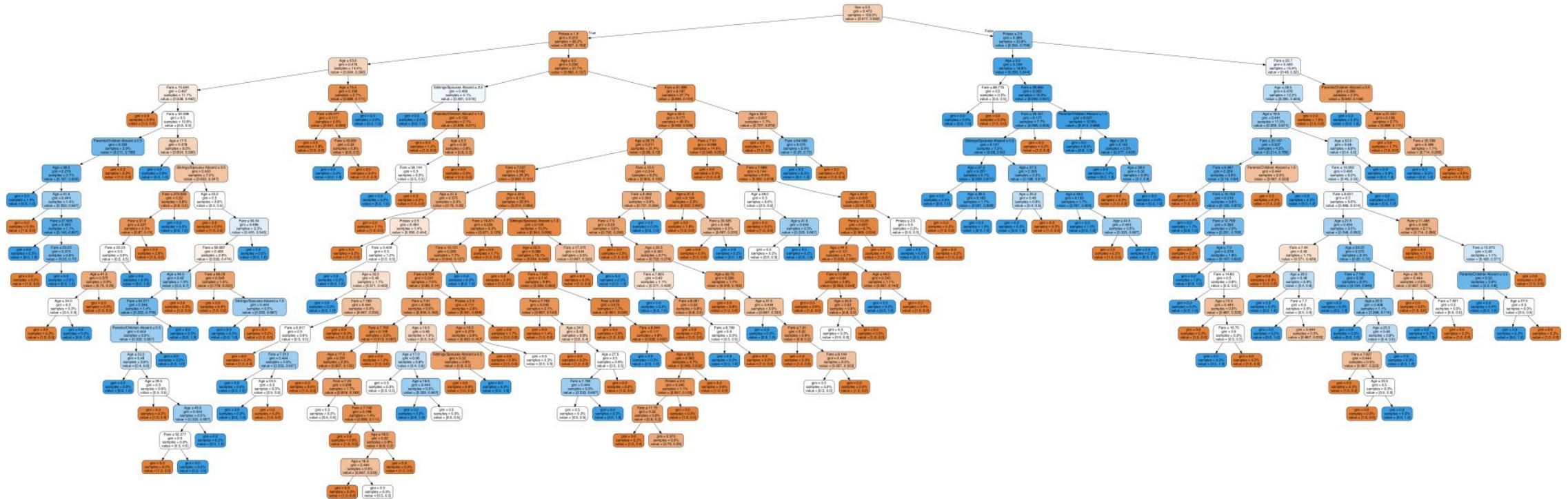
- The IG-based algorithm finds the smallest tree possible. Why?
- Occam's razor: prefer the ***simplest*** hypothesis that fits the data.

→ 다양한 케이스가 가장 확실하게 일반화가 되는 간단한 가설을 채택해야 일반화 성능이 최우선 목표



Another Potential Issue

- Decision trees are prone to over-fit to the training data.

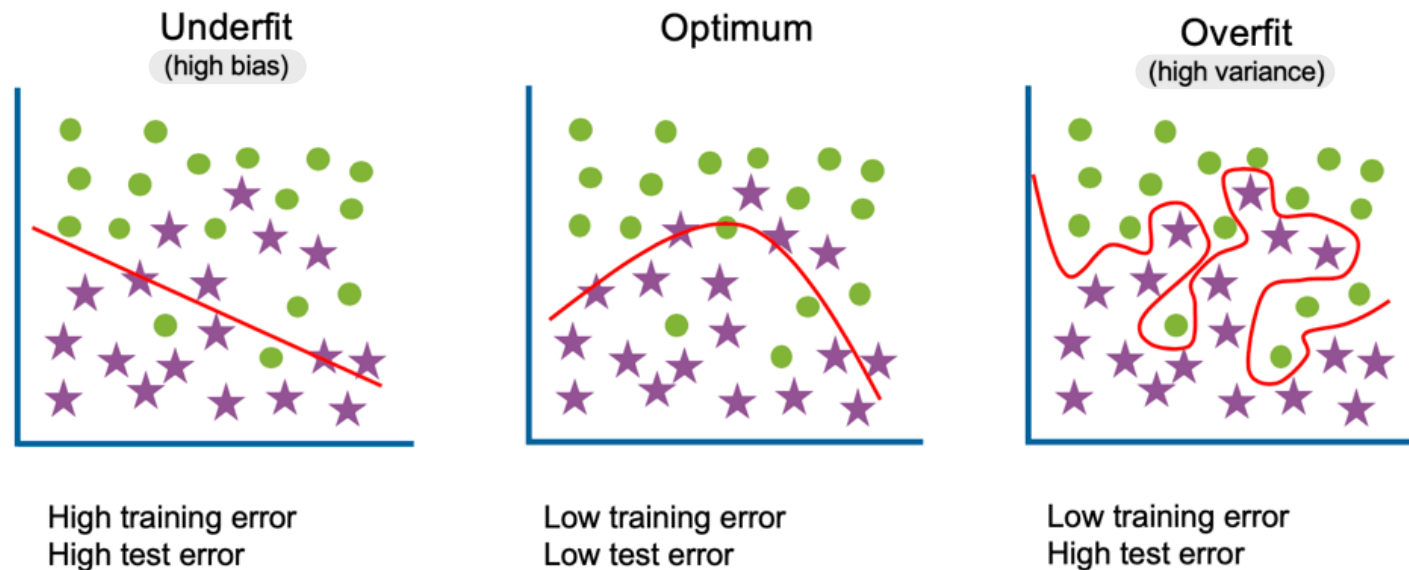


Maybe your model will be too specific...

- Decision Tree
- Building a Decision Tree
- ***Overfitting Issue***

Overfitting

- Overfitting is an undesirable machine learning behavior that occurs when the model gives accurate predictions for training data but not for new data.
 - It causes a large gap between the training accuracy and the validation accuracy.



Overfitting Formally

- Consider error of hypothesis h over
 - Training data: $error_{train}(h)$
 - Entire distribution D of data: $error_D(h)$
- Hypothesis $h \in H$ overfits training data if there is an alternative hypothesis $h' \in H$ such that

$$error_{train}(h) < error_{train}(h')$$

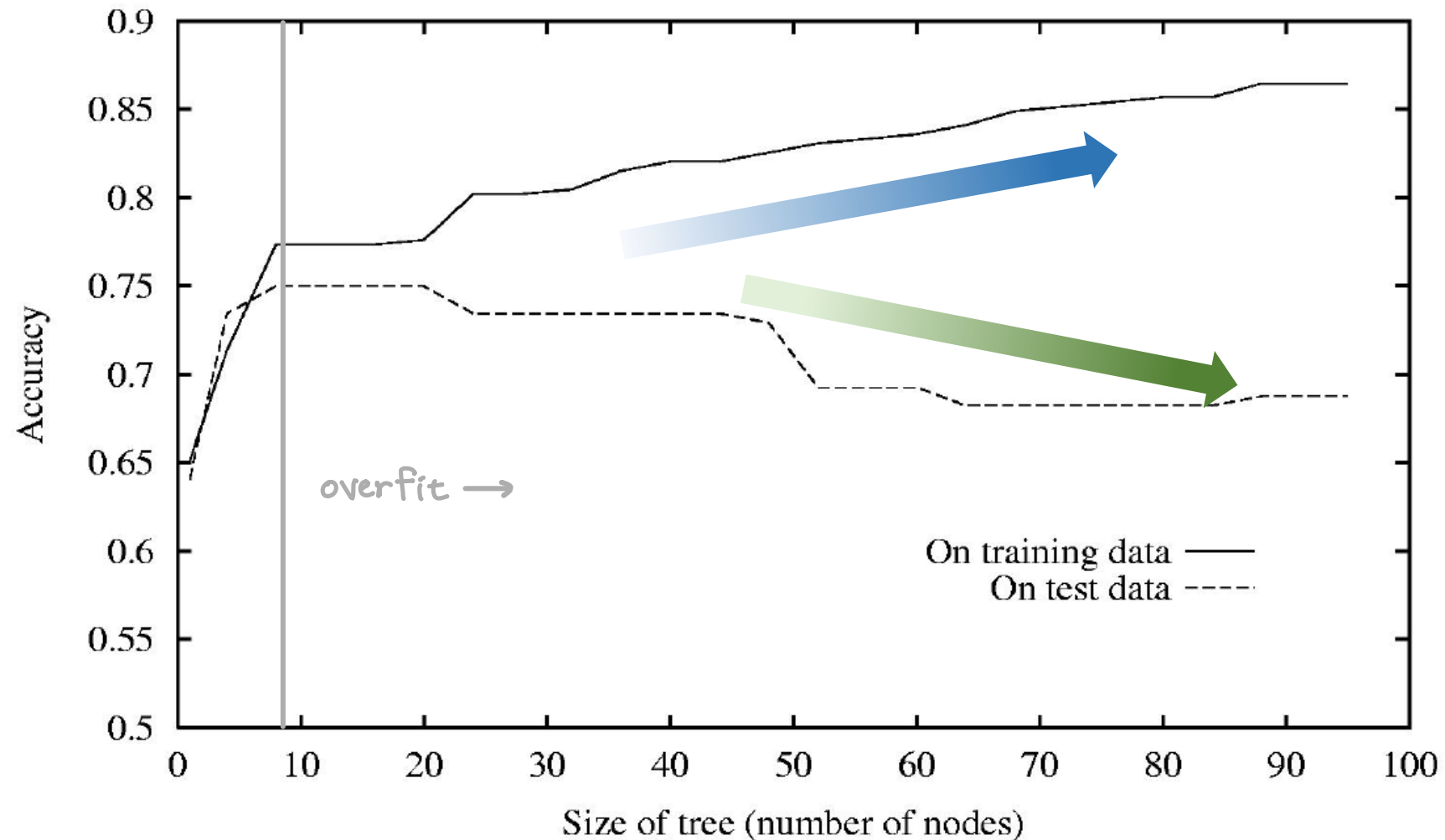
훈련 데이터 셋에 대해서
는 오류가 가장 낮음

and

$$error_D(h) > error_D(h')$$

전체 데이터 셋에 대해서는 오
류가 더 낮은 가설이 존재함

Example: Overfitting



What Makes Overfitting? (1/2)

- The training data size is too small and does not contain enough data samples to accurately represent all possible input data.



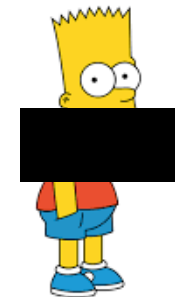
Do you know who this is?

- The training data contains large amounts of irrelevant information, called noisy data.

Bart Simpson



Bart Simpson

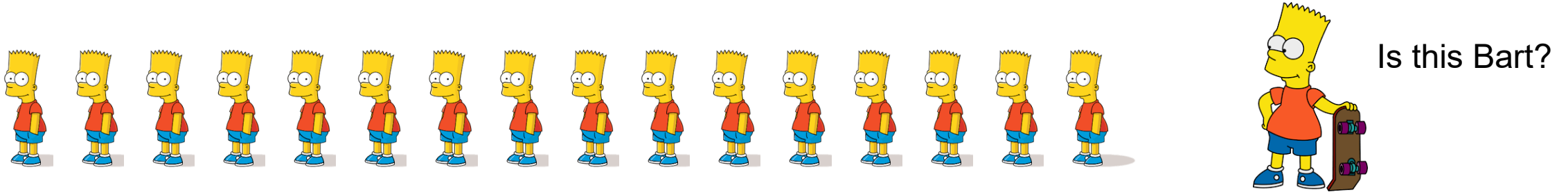


Bart Simpson

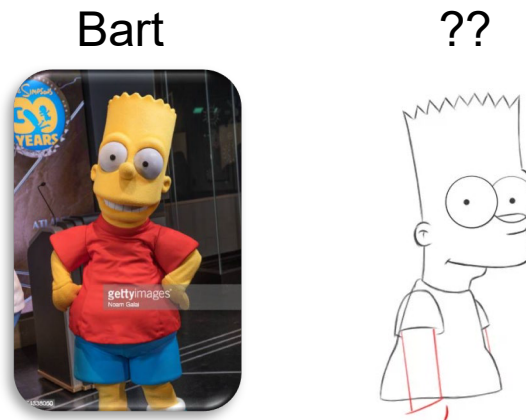


What Makes Overfitting? (2/2)

- The model trains for too long on a single set of data. $\approx$ bias



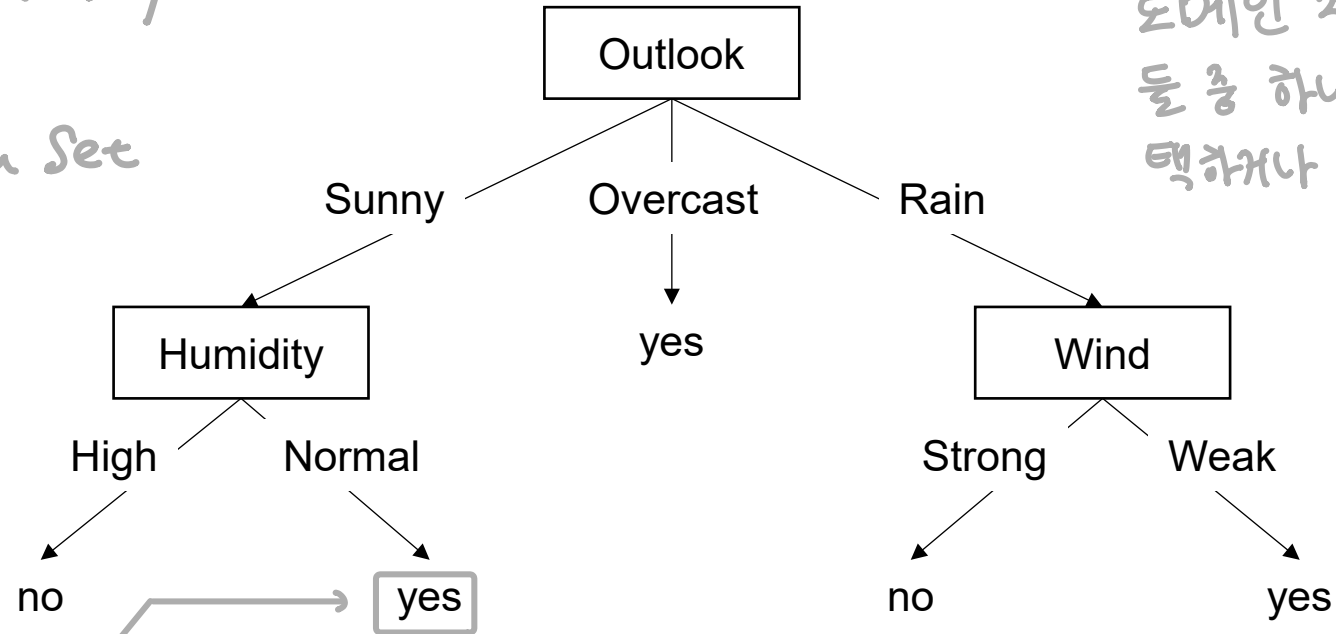
- The model complexity is too high, so it learns the noise within the training data.



Overfitting in Decision Trees

Data Inconsistency
or
Too Small Data Set

★ → 속성이 같으나 레이블이 다름 : 학습에 의미가 없으며
도메인 지식을 고려하여
들 중 하나의 레이블만 선
택하거나 더 많은 데이터 확보



What if we add a sample below?

<outlook=**sunny**, temperature=**hot**, humidity=**normal**, wind=**strong**, playTennis=**No**>

Avoiding Overfitting

- First, *collect more training data*...
 - This is the most effective solution. The more is always the better.
- Second, try your best to *clean the dataset* (remove irrelevant attributes).
- Finally, intentionally *penalize your model*. = 정규화
 - Stop growing your tree before it perfectly fits to your training data.
 - Alternatively, ***you can prune the tree afterward***.
 - Reduced-Error Pruning
 - Rule Post-Pruning

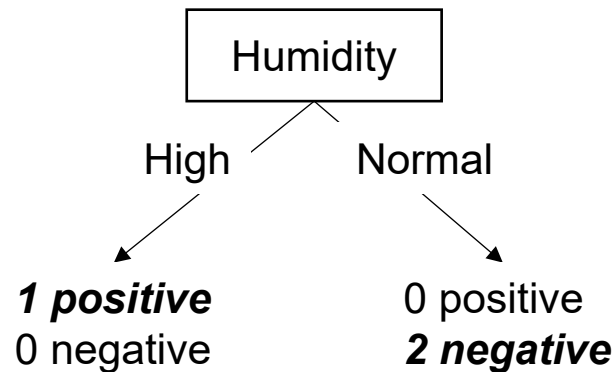
Reduced-Error Pruning (1/3)

1. Split data into training and validation sets.
2. Grow tree based on training set.
3. Traversing over all individual nodes (bottom-up), ***check the validation accuracy after removing each node***. If the accuracy remains the same or even gets better, remove the node.
4. Repeat the above step until any further pruning harms the accuracy.

Reduced-Error Pruning (2/3)

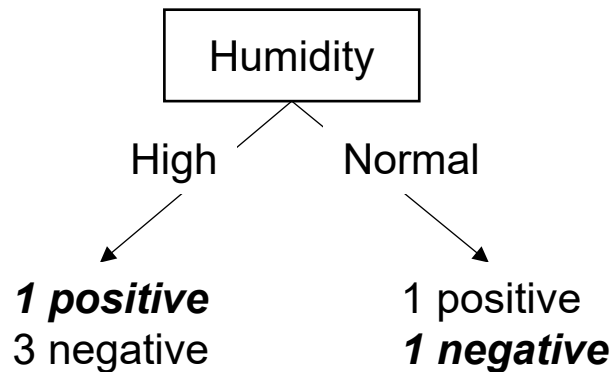
- Replace a subtree only when the expected error of the subtree is greater than that of having a single leaf node.
- Example:

Training data



3 corrects and 0 wrongs, perfect!

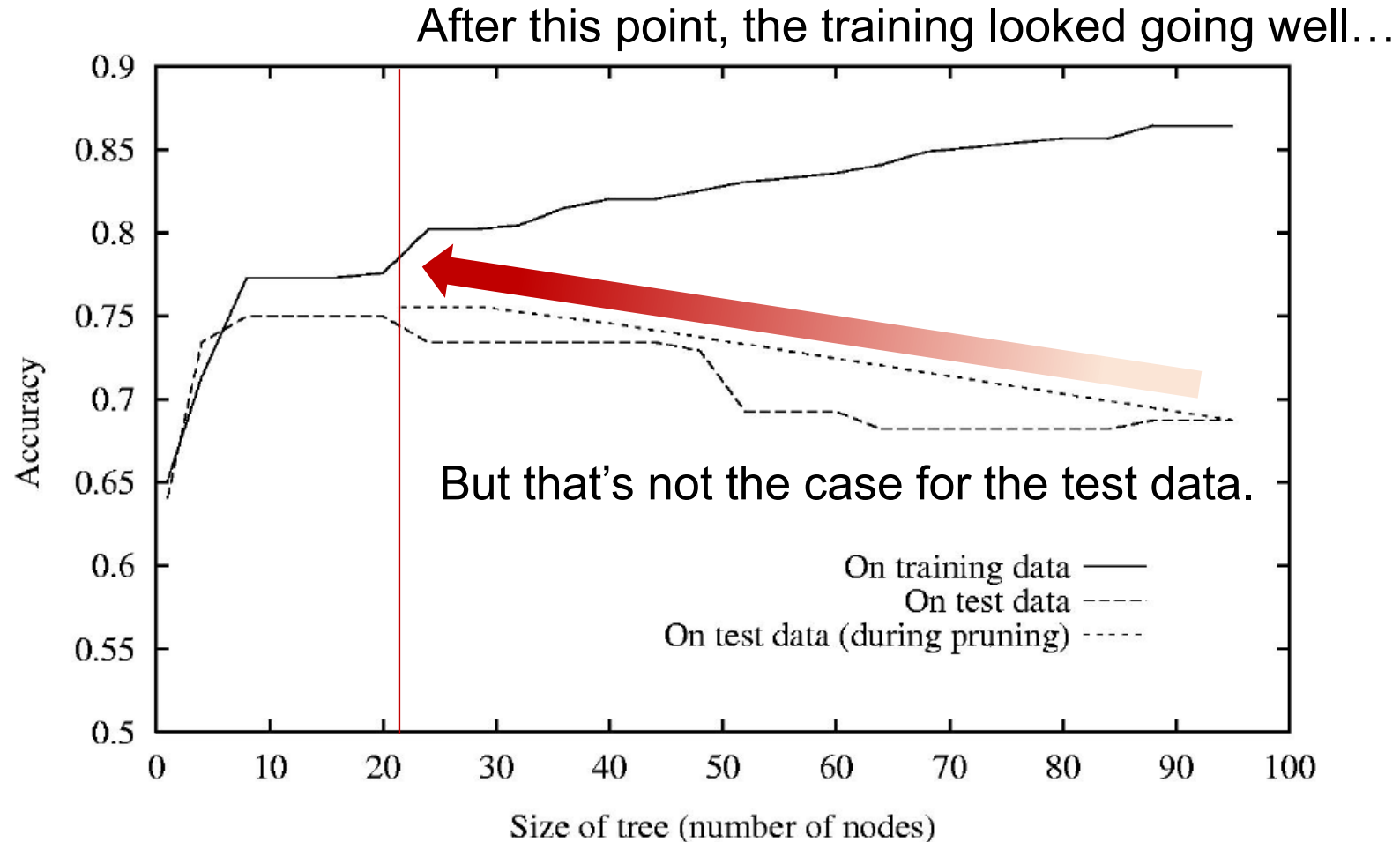
Validation data



2 corrects and 4 wrongs...

If this humidity attribute is replaced with a single leaf node of **negative**, only 2 wrongs and 4 corrects!

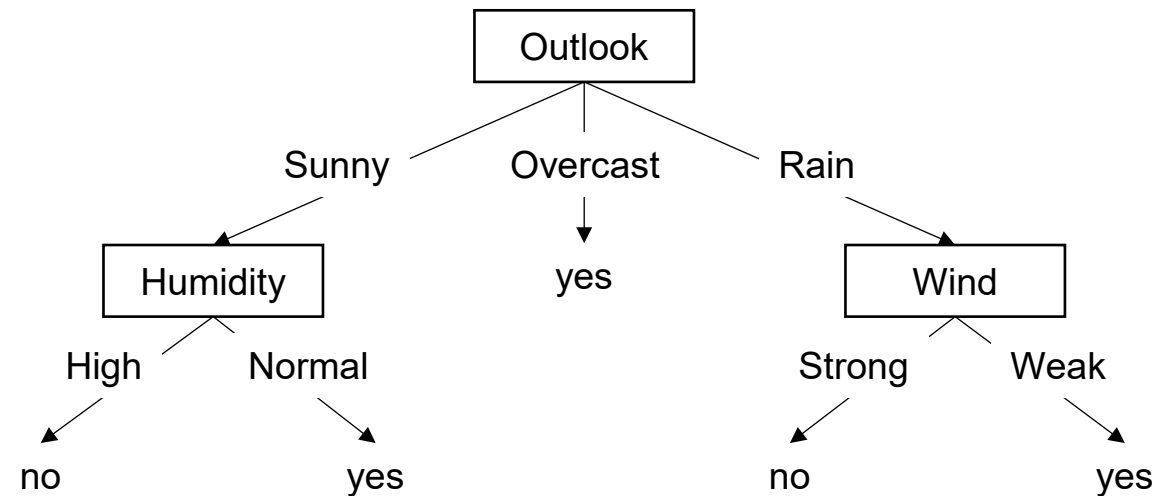
Reduced-Error Pruning (3/3)



The tree is pruned back to the red line where it gives higher test accuracy.

Converting a Tree to Rules

- A rule is derived from each path from the root to a leaf.



If (outlook=sunny) and (Humidity=high) then playTennis=no

If (outlook=sunny) and (Humidity=low) then playTennis=yes

...

Rule Post-Pruning

1. Fully train a tree on the training data first. Bottom Up이 아닌 전체 트리에
대해서 Path별로 탐색 및 삭제
2. Convert the tree to a equivalent set of rules.
3. For each rule, check if any preconditions harm the validation accuracy. If yes, prune the rule.
 - E.g.: choose the best one only among the below.
 - (outlook=sunny) then playTennis=no
 - (Humidity=high) then playTennis=no
 - (outlook=sunny) and (Humidity=high) then playTennis=no
4. Finally, sort all the remaining rules by their validation accuracy.

Rule Post-Pruning (2/2)

- **Pros:**

- (Flexibility) In the reduced-error pruning, if you prune a node, the whole branches are removed. In contrast, the rule post-pruning can remove specific attributes on all individual paths!
- It improves interpretability.

└─> 설명가능성 높음, 대부분 ML 기법은 설명가능성 낮음

- **Cons:**

- Since all the paths and all individual attributes of them should be checked separately, the cost is much higher. 계산량 많음

Overfitting is General Issue in ML

- Not only decision trees but also many other ML algorithms suffer from overfitting.

- Linear/logistic regression
- SVM
- Neural Networks



- Pruning can be considered as a special type of ‘regularization’.
- We will discuss the regularization more deeply soon!

풀기보다는 파라미터 튜닝으로 적당하게 극복하는 게 더 합리적, 머신러닝으로는 완전히 해결하는 것이 불가능

Any Questions?

Now you know decision trees too!

